

Полное руководство по Git, настройке Termux и процессам

1. Настройка Termux и автоматизация (.bashrc)

Файл `~/.bashrc` отвечает за команды, которые автоматически выполняются при каждом запуске Termux. Мы добавили туда поддержку автодополнения Git и быстрое сокращение для очистки экрана.

Как теперь выглядит ваш блок в конце `~/.bashrc` :

```
# Включение автодополнения (кнопка Tab) для команд Git и Linux
source /data/data/com.termux/files/usr/share/bash-completion/bash_completion

# Быстрое сокращение для очистки экрана терминала
alias clr="clear"
```

Как исправить старую ошибку ualias:

Если при запуске Termux пишет `No command ualias found`, исправьте опечатку в первой строке файла одной командой:

```
sed -i 's/ualias/alias/g' ~/.bashrc
```

После этого примените настройки: `source ~/.bashrc`

2. Краткий справочник по Git (Ваш личный Git-h)

Команда	Что делает на понятном языке
<code>git status</code>	«Главный радар». Показывает, в какой вы ветке и какие файлы были изменены, удалены или созданы.
<code>git config --global core.quotepath false</code>	Исправление кодировки. Включается один раз, чтобы Git красиво показывал русские буквы в именах файлов вместо цифр.
<code>git diff</code>	«Цифровое увеличительное стекло». Показывает построчно, что именно изменилось в файлах. Выход из режима — кнопка <code>Q</code> .
<code>git checkout -b имя_ветки</code>	Создать черновик. Создает новую локальную ветку на телефоне и сразу переключает вас в неё.
<code>git switch имя_ветки</code>	Переключение каналов. Прыжок между ветками. Меняет файлы на экране за долю секунды.
<code>git mv старый_путь новый_путь</code>	Умный переезд. Переносит файлы в новые папки так, чтобы Git мгновенно понял перемещение и не терял историю.
<code>git commit -m "описание"</code>	Точка восстановления. Замораживает и сохраняет ваши подготовленные изменения в локальную историю с вашим комментарием.
<code>git push</code>	Отправка в облако. Выталкивает ваши сохраненные коммиты на сервер GitHub.

3. Разбор практического урока по уборке (Ветки и Папки)

Аналогия: Ветка `main` — это чистовая парадная комната. Ветка `cleanup` — изолированная мастерская. Путь проекта `~/.../downloads/bot_android_adb` при переключении веток не меняется, Git просто мгновенно меняет «декорации» (файлы) на лету.

Последовательность действий, которую вы успешно выполнили:

1. Проверили чистоту перед уборкой: `git status`
2. Создали черновик и ушли в него: `git checkout -b cleanup`
3. Выгрузили ветку на GitHub впервые: `git push -u origin cleanup`
4. Создали новую папку для документов: `mkdir docs`
5. Перенесли туда файл: `git mv "запуск SSH на устройстве.txt" docs/`
6. Зафиксировали уборку коммитом: `git commit -m "Создал папку docs и перенес инструкцию по SSH"`
7. Отправили изменения в облако: `git push`

4. Руководство по кнопке Tab и отсечению процессов (Handoff)

Суперсилы кнопки Tab:

- **Дописание команд:** Наберите `git stat` и нажмите **Tab** — консоль сама допишет до `git status`.
- **Автодополнение папок и файлов:** Наберите `cd d` + **Tab** — терминал подставит `cd docs/`. Если вариантов несколько, нажмите **Tab дважды** для вывода списка.
- **Экранирование пробелов:** Если в названии файла есть пробелы, Tab автоматически добавит кавычки или обратные косые черты (слеши), чтобы защитить команду от ошибок.

Экстренное завершение (Убийство) зависших процессов:

- `Ctrl + C` — главная кнопка спасения. Моментально обрывает и намертво убивает любой зациклившийся скрипт (например, бота или кликера), который завис на экране.
- `Ctrl + Z` — ставит зависший процесс на паузу и убирает его в фоновый режим, освобождая строку ввода.
- `kill python` — если терминал намертво «залип» и не реагирует на клавиши, откройте новую сессию (свайп слева) и введите эту команду. Она принудительно закроет все запущенные скрипты Python на телефоне.